SCHOOL MANAGEMENT SYSTEM

Unit 2 Project Documentation

Student Name: Ghadah Ammar Alabbadi
Programming Language: Python
Project Type: Command-Line School Management System

1. Project Abstract

The School Management System is a Python command-line application developed to manage the main academic records of a school. The system manages students, teachers, and courses through a menu-driven interface.

It supports adding and removing records, enrolling and unenrolling students in courses, assigning courses to teachers, and displaying stored information. The project uses object-oriented programming to separate responsibilities between Student, Teacher, Course, and School classes.

Data persistence is implemented using JSON files so that student, teacher, and course information can be restored when the application starts. Input validation is handled through a dedicated validators module, while exceptions are caught in the main program and recorded in a log file. The design therefore combines object-oriented structure, file handling, validation, error handling, and logging in one small management system.

2. System Design Documentation

The system was designed before implementation using three UML views. Each diagram presents the system from a different perspective.

2.1 Class Diagram

The class diagram focuses on the School class because it acts as the central manager of the application. It stores dictionaries of students, courses, and teachers and provides the operations that connect these objects, such as enrollment, deletion, searching, and course assignment.

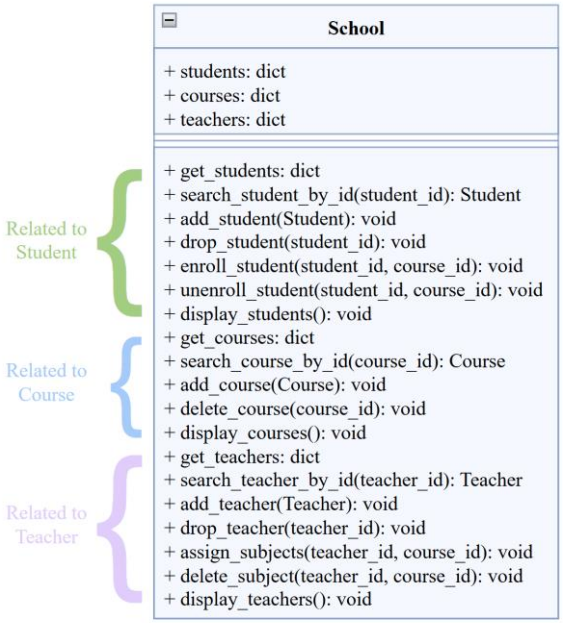


Figure 1 Class School

2.2 Use Case Diagram

The use case diagram represents the School Administrator as the main actor. The administrator can manage students, courses, and teachers and can save and exit the application. Included use cases represent required validation or supporting actions, while extended use cases represent error-handling paths that occur only when a condition fails.

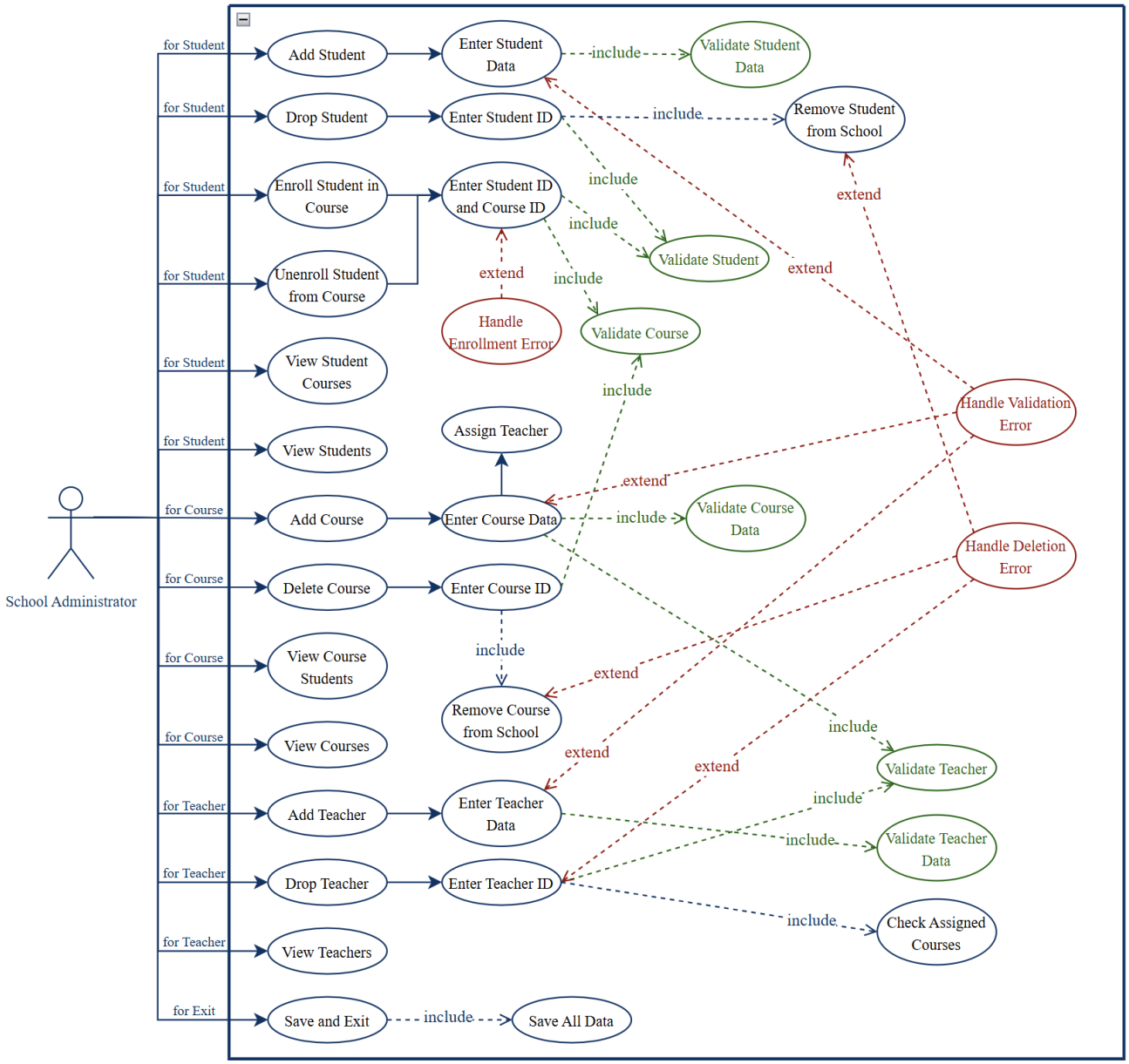


Figure 2 Use Case Diagram

2.3 Activity Diagram

The activity diagram documents the enrollment process. The system receives a student ID and course ID, validates the records, checks duplicate enrollment and course capacity, updates both related objects, saves the changes, and then displays a confirmation. Invalid conditions lead to an error path instead of updating the records.

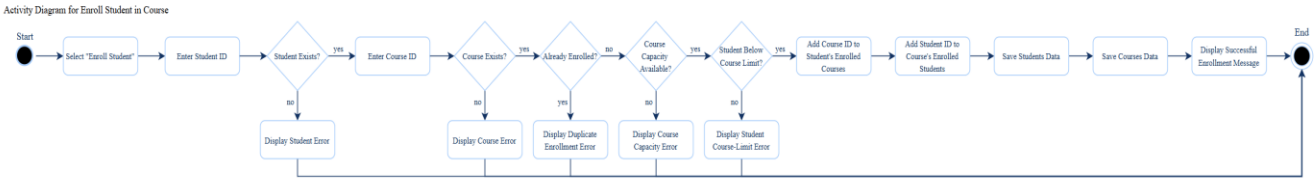


Figure 3 Enrollment Activity Diagram

3. Code Documentation

3.1 Project Structure

The source code is separated into models, utilities, data files, and the main application. This keeps the responsibilities of the project clear and makes the code easier to maintain.

```
school_system/
├── main.py
├── models/
│   ├── school.py
│   ├── student.py
│   ├── course.py
│   └── teacher.py
├── utils/
│   ├── validators.py
│   ├── file_handler.py
│   └── logging_handler.py
├── data/
│   ├── students.json
│   ├── courses.json
│   └── teachers.json
└── log.txt
```

Figure 4 Project Structure

3.2 Main Program

The main.py file is the entry point of the application. It creates one School object, loads previously saved JSON data into model objects, displays the menu, receives the user choice, calls the required model methods, and saves successful changes back to the JSON files.

```
STUDENTS_FILE = "data/students.json"
COURSES_FILE = "data/courses.json"
TEACHERS_FILE = "data/teachers.json"
SCHOOL_OBJECT = school.School()
```

Figure 5 Data-file constants and the main School object

```
while True:

    display_menu()

    try:
        user_choice = input("Choose an option: ")
        match user_choice: ...
    except Exception as e:
        print(e)
        logging_handler.log_error(str(e))
```

Figure 6 Main application control flow

3.3 Student Model

The Student class stores the student ID, name, age, and the list of enrolled course IDs. A student can enroll in a maximum of seven courses. The class also prevents duplicate registration and supports removing a course from the student record.

```
class Student:

    def __init__(self, student_id, name, age):
        self.student_id = student_id
        self.name = name
        self.age = age
        self.enrolled_courses = []


    @property
    def get_enrolled_courses(self): ...


    def enroll_to_course(self, course_id):
        """Method for enroll student to specific course in Student"""
        if len(self.enrolled_courses) < 7:
            if course_id not in self.enrolled_courses:
                self.enrolled_courses.append(course_id)
            else:
                raise ValueError("Student is already registered for this course")
        else:
            raise ValueError("Student reached the maximum number of course registrations, which is 7")


    def unenroll_to_course(self, course_id): ...
```

Figure 7 Student initialization and enrollment logic

3.4 Course Model

The Course class stores the course ID, title, assigned teacher ID, and enrolled student IDs. The course capacity is limited to 25 students. The add_student and remove_student methods maintain the relationship from the course side.

```python
class Course:

    def __init__(self, course_id, title, teacher_id):
        self.course_id = course_id
        self.title = title
        self.teacher_id = teacher_id
        self.enrolled_students = []


    @property
    def get_enrolled_students(self): ···


    def add_student(self, student_id):
        """Method for add specific student to enrolled_students list in Course"""
        if len(self.enrolled_students) < 25:
            if student_id not in self.enrolled_students:
                self.enrolled_students.append(student_id)
            else:
                raise ValueError("Student is already registered")
        else:
            raise ValueError("Course reached its maximum student capacity of 25")


    def remove_student(self, student_id): ···
```

Figure 8 Course data and student-capacity logic

3.5 Teacher Model

The Teacher class stores the teacher ID, name, and a list of assigned course IDs in subjects. A teacher can be assigned to a maximum of six courses. The class provides methods to add or remove a course assignment.

```python
class Teacher:

    def __init__(self, teacher_id, name):
        self.teacher_id = teacher_id
        self.name = name
        self.subjects = []


    def add_subject(self, subject):
        """Method for add specific subject to teacher in Teacher"""
        if len(self.subjects) < 6:
            self.subjects.append(subject)
        else:
            raise ValueError("Teacher reached the maximum number of courses")


    def drop_subject(self, subject): ···
```

Figure 9 Teacher data and subject-assignment limit

3.6 School Model

The School class coordinates the other model objects. It stores students, courses, and teachers in dictionaries where IDs are used as keys. It is responsible for searching, adding, deleting, displaying, enrollment, unenrollment, and maintaining relationships when records change.

```
def enroll_student(self, student_id, course_id):
    """Method for enroll specific student to specific course in School"""
    student = self.search_student_by_id(student_id)
    course = self.search_course_by_id(course_id)
    if len(course.enrolled_students) >= 25:
        raise ValueError("Course reached its maximum student capacity of 25")
    student.enroll_to_course(course_id)
    course.add_student(student_id)
```

Figure 10 School method that coordinates student enrollment

```
def delete_course(self, course_id):
    """Method for delete specific course from courses dict in School"""
    for student in self.students.values():
        if course_id in student.enrolled_courses:
            student.unenroll_to_course(course_id)
    self.courses.pop(course_id)
```

Figure 11 Course deletion and cleanup of student records

3.7 Data Persistence

The application stores data in three JSON files: students.json, courses.json, and teachers.json. The file_handler module centralizes JSON reading and writing. Missing files or invalid JSON content return an empty dictionary, allowing the application to start without crashing.

```
def write_to_json(file_path, data):
    with open(file_path, "w", encoding="utf-8") as file:
        json.dump(data, file, indent=2)
```

```
def read_json(file_path):
    try:
        with open(file_path, "r", encoding="utf-8") as file:
            data = json.load(file)
            return data
    except (FileNotFoundError, json.JSONDecodeError):
        return {}
```

Figure 12 JSON file-handling functions

3.8 Validation and Exception Handling

Input validation is separated into validators.py. The module validates text, age values, IDs, and duplicate course titles. Invalid input raises ValueError with a clear message. These exceptions are handled by the main program, which prevents the application from stopping unexpectedly.

```python
def validate_number(number):
    """Function for validate Integer input in validators file"""
    number = number.strip()
    if not number:
        raise ValueError("Field can't be empty")
    elif not number.isdigit():
        raise ValueError("Field must be a number")
    elif int(number) < 5 or int(number) > 65:
        raise ValueError("Age must be greater than 4 and less than 66")
    else:
        return int(number)
```

Figure 13 Validate number function

3.9 Logging

Errors caught in main.py are passed to logging_handler.py. Each log entry contains the date and time, the ERROR level, and the exception message. The file is opened in append mode so previous logs are preserved between program runs.

```python
from datetime import datetime


LOG_FILE = "log.txt"


def log_error(message):
    with open(LOG_FILE, "a", encoding="utf-8") as file:
        file.write(f"{datetime.now()} | ERROR | {message}\n")
```

Figure 14 Error logging implementation

4. Conclusion

The School Management System demonstrates the use of object-oriented programming in a practical command-line application. The final design separates model responsibilities, maintains relationships between students, courses, and teachers, stores records in JSON files, validates user input, and logs errors. The project structure makes the system easier to understand and provides a clear base for future improvements or additional features.